

DT-as-a-Service on the RADICAL stack

Architecture · implementation · use-case mapping · August 2026
RADICAL, Rutgers University

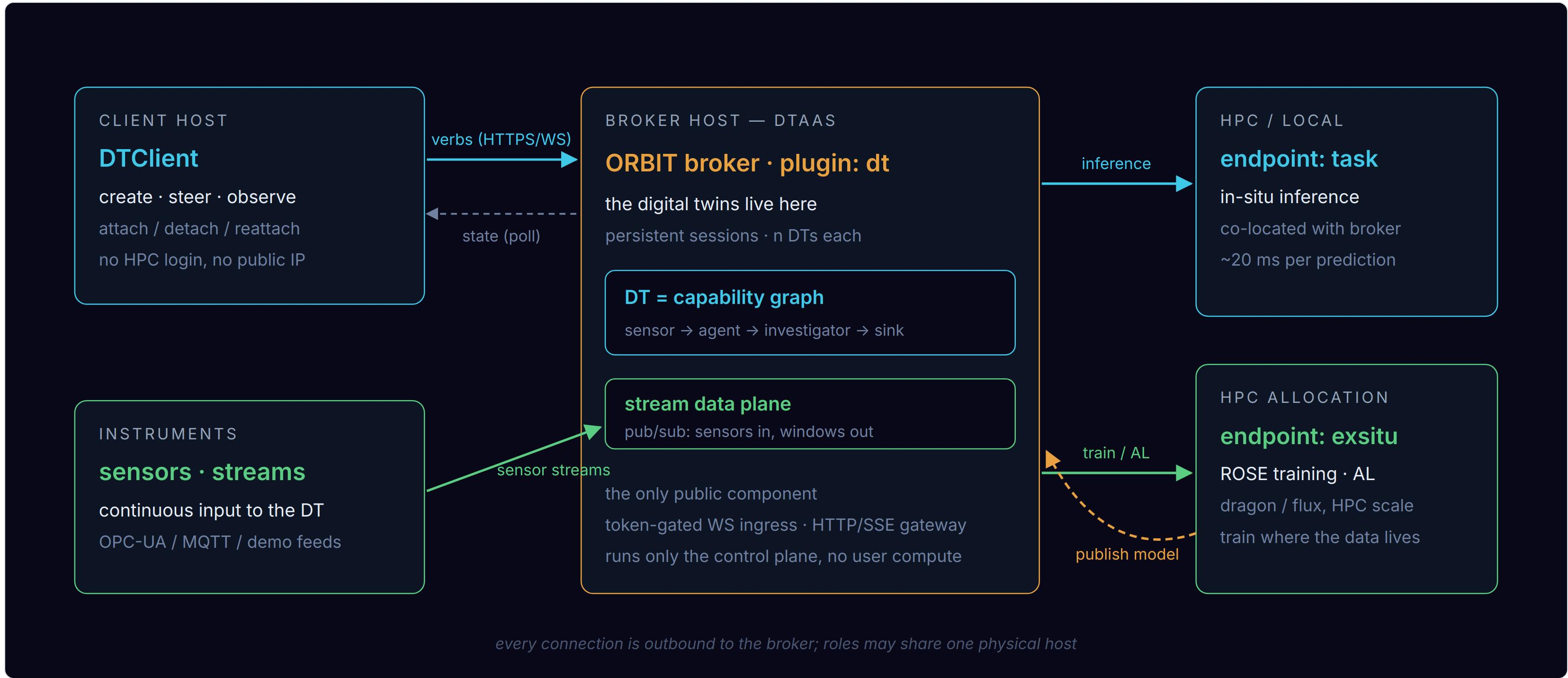
*Digital twins as long-running services on DOE HPC.
v1 is implemented and measured.*

From framework to service

- ▶ The DT **framework** defines the digital twin: a capability graph whose models **keep learning** while **serving inference**.
- ▶ A digital twin outlives any interactive session: the inference loop runs **indefinitely**, learner loops span **many allocations**.
- ▶ **DTaaS**: the digital twin's control plane runs as an ORBIT plugin on a **standalone, persistent broker**.
- ▶ One broker hosts **multiple sessions** (name spaces); each session holds **multiple DTs**. The client is a **console**: attach, steer, detach, while the DT keeps running.
- ▶ Digital twins are defined **programmatically** in v1 (builder API); the declarative YAML spec comes later, cast over the proven API.

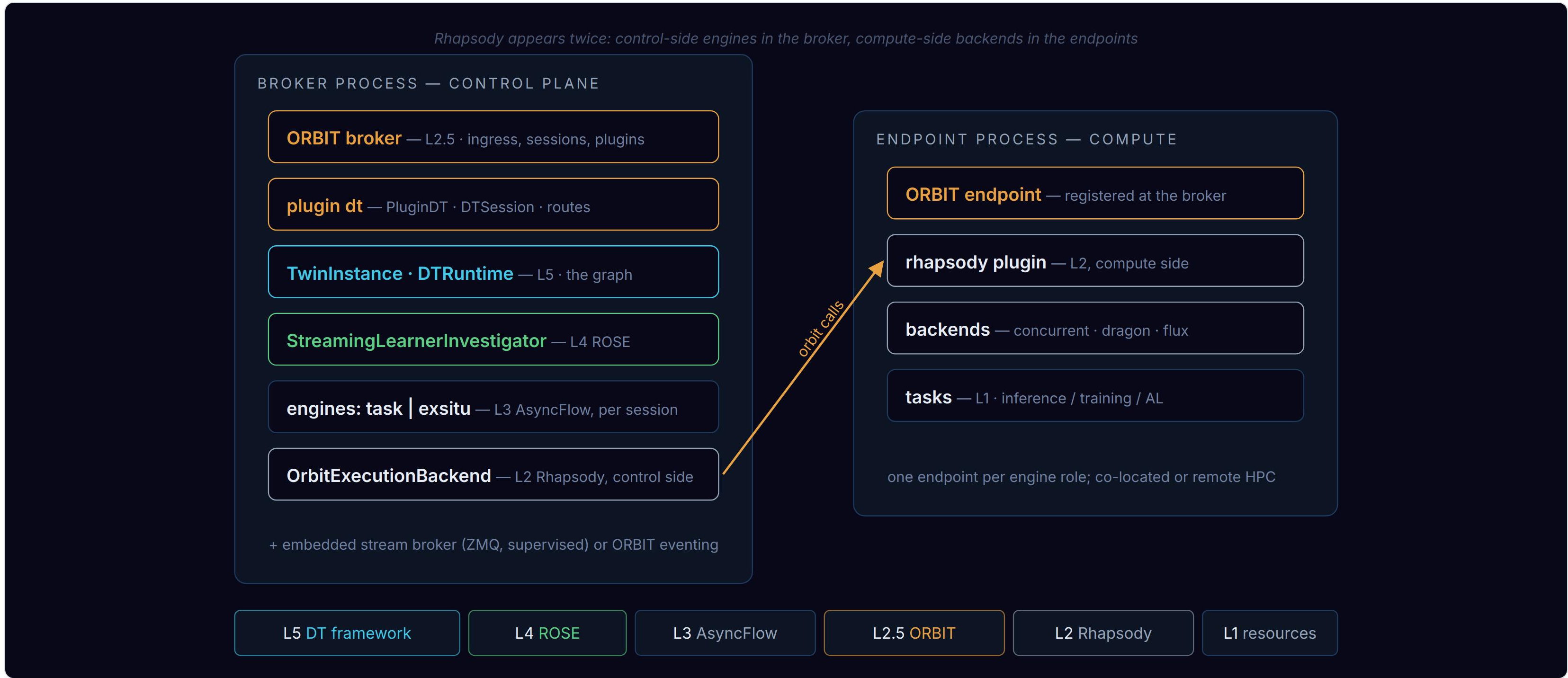
L5 DT framework → L4 ROSE → L3 AsyncFlow → **L2.5 ORBIT** → L2 Rhapsody → L1 resources

Architecture — roles



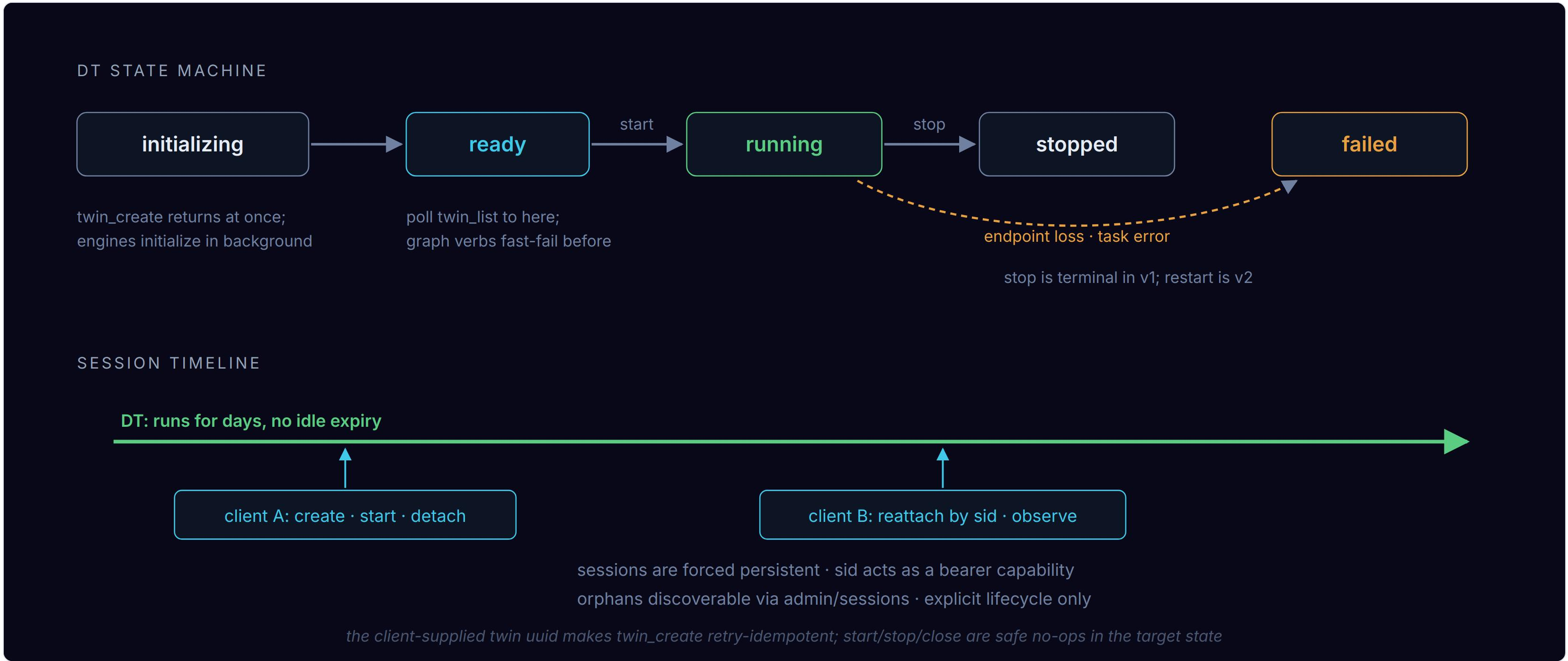
Four roles: the client steers, instruments feed, the broker hosts the digital twins, endpoints compute. The broker is the only publicly reachable component.

Architecture — software



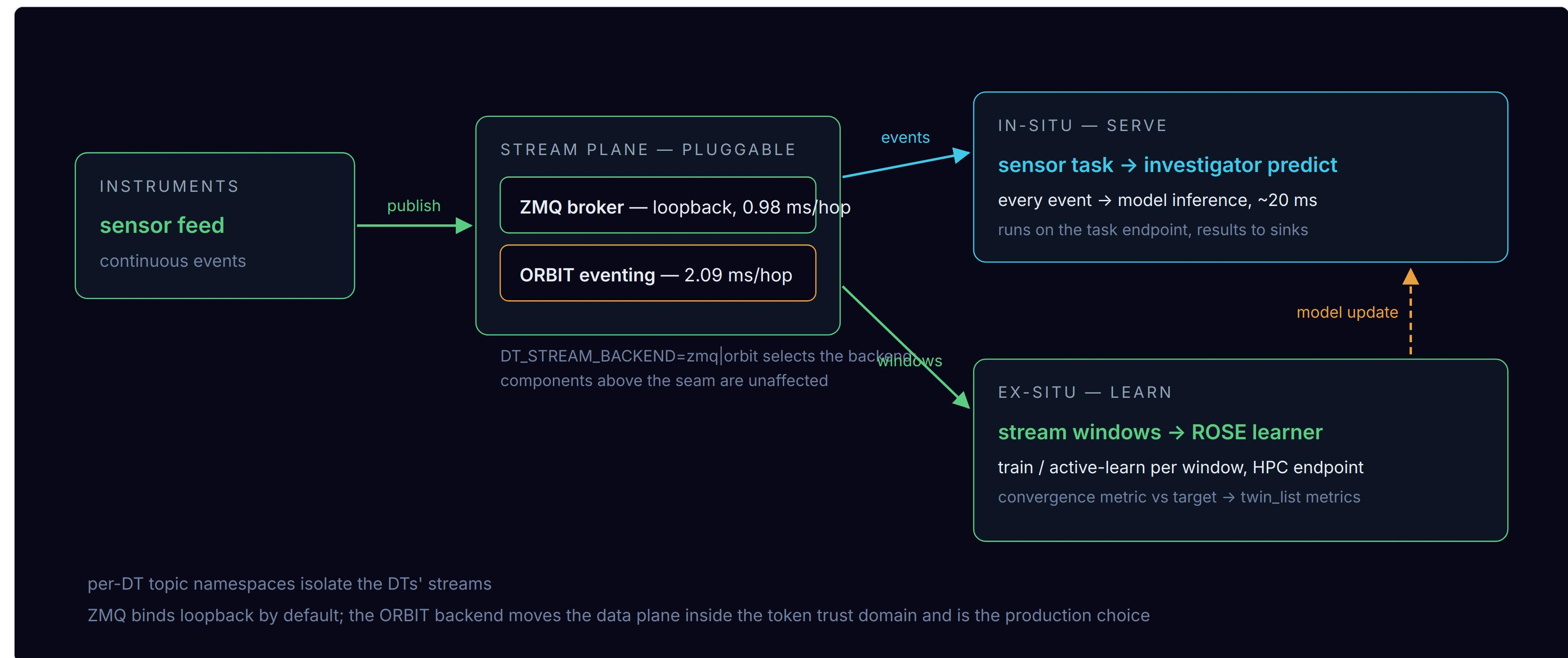
New v1 code is the dt plugin and the hardened L5 runtime; the layers below are reused unchanged.

Lifecycle — DT states and sessions



twin_create is the only asynchronous verb; all other verbs are short and synchronous. Clients attach and detach; the digital twin continues to run.

Data plane — streams and learning



One stream feeds two loops: serving runs on every event, learning on every window; the model update closes the learning cycle.

Implementation status

MILESTONE	DELIVERS	STATE
M0 hardening	stop/teardown, per-DT topic namespaces, embeddable stream broker, packaging, pytest/tox	PR ready
M1 ORBIT plugin	dt plugin: persistent sessions, sid reattach, async twin_create + short sync verbs, admin listing	PR ready
M2 ROSE ex-situ	StreamingLearnerInvestigator, dual-engine DTs (task exsitu), endpoint-loss visibility	PR ready
M3 data plane	ORBIT-pubsub backend: stream traffic inside the token domain	PR ready

- ▶ 126 tests, including live-broker integration. Demos verified with both stream backends: service DT and streaming learner; calibration converges.
- ▶ Upstream: two ORBIT PRs merged; one AsyncFlow fix + one ROSE dependency in review.
- ▶ Live viz dashboard: convergence bars and activity lanes, in the ORBIT Explorer and standalone, with live and replay modes.

Measured performance

MEASUREMENT	RESULT	DECISION IT SETTLED
in-situ prediction	~20 ms p50 via endpoint vs ~11 ms in-process; endpoint wins under concurrency	all compute goes through Rhapsody, without an in-process pool; a co-located endpoint is the "local" deployment
stream hop	0.98 ms ZMQ · 2.09 ms ORBIT	the secure ORBIT data plane costs ~1 ms per hop and becomes the production default
sequential latency	261 ms → 20 ms after fixing the rhapsody notify batcher	fixed upstream as an endpoint configuration knob

Benchmarks live in the repo (perf/) and rerun against the demos.

Map to use-case requirements

REQUIREMENT (AMSC · XGFABRIC)	DTAAS V1 ANSWER
DT outlives any login session	persistent sessions; DTs survive clients, reattach by sid
drive from a laptop	outbound-only ORBIT connectivity; no HPC login, no public IP
train where the data lives	exsitu engine → Rhapsody endpoint in the HPC allocation (dragon / flux)
continuous sensor streams	stream plane with per-DT namespaces; ZMQ or ORBIT backend
serve inference continuously	in-situ loop on the task endpoint, ~20 ms per prediction
DOE-site security posture	token-gated ingress; M3 puts stream traffic in the same trust domain
many DTs, many users	n DTs per session, admin listing; per-tenant auth is v2
model selection · capability graphs · recovery	v2 : cross-model selector, declarative graph spec, restart across broker loss

Next

- ▶ **Merge path:** the four stacked PRs are under review; upstream AsyncFlow / ROSE merges unpin the last dependencies.
- ▶ **First real DTs:** the MATEY fine-tuning stream and the xGFabric wind pipeline (after a smoke pass).
- ▶ **v2 candidates:** DT recovery/restart, per-tenant auth, capability-graph spec, scale-out (endpoint-hosted DTs).

Plan and measurements: [digital_twins/docs/dtaas-v1-plan.md](#) · architecture: [amsc/architecture/dt-framework.md](#)